

Process:-

A process is a program under execution. The components of a process are: the program to be executed, the data on which the program will execute, the resources required by the program - such as memory and files - and the status of execution.

Program resides in secondary memory where as process resides in main memory. Both are executable but the program is passive and process is active.

Some systems allow execution of only one process at a time (eg: early computers). They are called uni programming systems. Others allow more than one process, i.e., concurrent execution of many processes. They are called multi programming (not multi processing) systems.

In multi tasking system, the CPU switches automatically from process to process running each for tens or hundreds of milliseconds, (based on time slice).

In reality the CPU is actually running one and only one process at a time.

Over years, O.S. designers evolved a model that makes concurrency easier to deal with. In this model, each runnable sits on the computer. Often components of the O.S. itself - is organised into a no. of (sequential) processes, each viewed as a block of code with pointer showing the next instruction to be executed.

How can several processes share one CPU? o.s. takes care of this by making sure:

- each process gets a chance to run - fair scheduling
- they do not modify each other's state - protection

Process description - The o.s. must know specific information about processes in order to manage and control system. Such information is usually grouped into two categories:

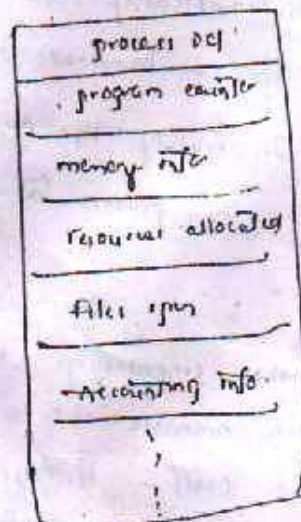
* Process state information

- eg: CPU registers (general and special purpose), program counter.

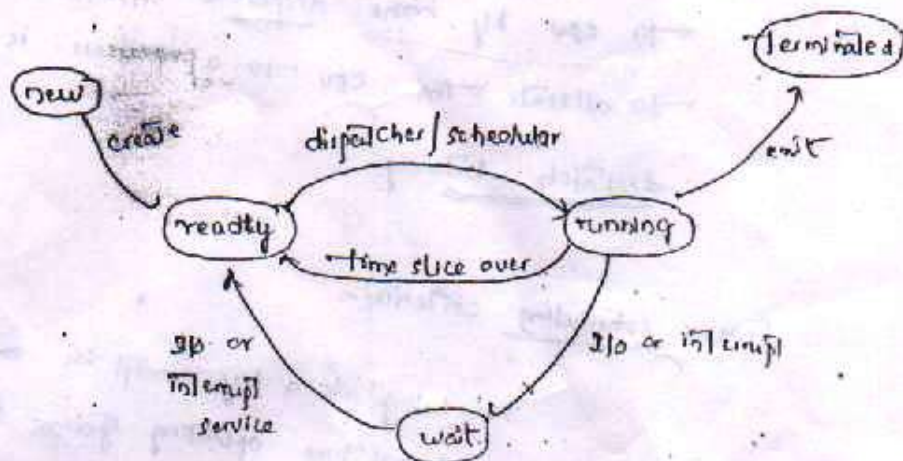
* Process control information

- eg: scheduling priority, resources held, access privileges, memory allocated, accounting.

This collection of process information is kept in access through a Process control Block (PCB).



PCB



Process state Diagram

There are no. of states that can be attributed to a process. Indeed, the operation of a multiprogramming system can be described by a state transition diagram on the process states. The states of a process include:

- **New**:- a process being created but not yet included in the pool of executable processes (resource acquisition) (ready queue)
- **Ready**:- processes that are prepared to execute when given opportunity
- **Running**:- the process that is currently being executed by the CPU
- **Blocked**:- the a process that cannot execute until some event occurs.
- **Terminate**:- a process that is about to be removed from the pool of executable processes (resource release)

The process selected by scheduler will be allocated to CPU by the dispatcher. The time taken to allocate the CPU to a process is called as dispatch latency.

Process scheduling criteria:-

Scheduling is a key concept in computer multitasking, multi processing and real time operating system design. In modern O.S., there are typically many more processes running than there are CPUs available to run them. Scheduling refers to the way processes are assigned to run on the available CPUs. This assignment is carried out by the known as a scheduler or is sometimes referred to as dispatcher.

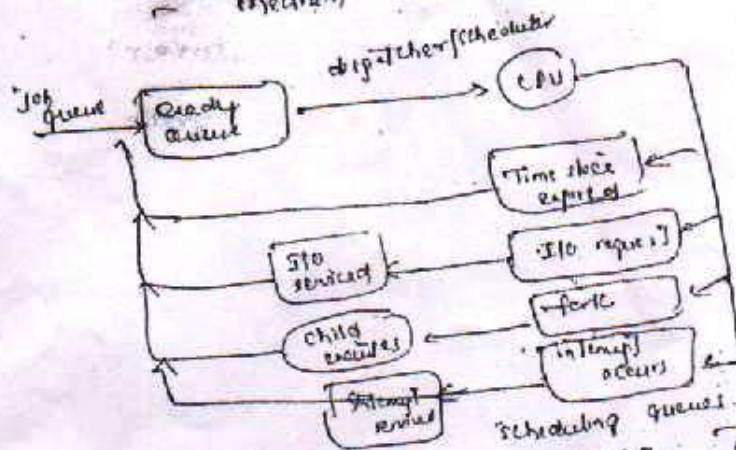
The scheduler is concerned mainly with:

- * CPU Utilization - to keep the CPU as busy as possible
- * Throughput - is a parameter to measure the performance of CPU. It is defined as total processes that are completed / executed per unit time.
- * Turnaround time - the time difference between job submission and job completed.
- * Waiting Time - amount of time a process has been waiting in the ready queue.
- * Response Time - amount of time it takes from when a request was submitted until the first response is produced.
- * Fairness - Equal CPU time to each thread.

There are Three types of schedulers :

23

- long term scheduler
- short term scheduler
- medium term scheduler.



It is the responsibility of LTS to place all the processes in ready queue. The process while placing them in a ready queue it will done process on means combination of both CPU burst and I/O burst.

The process that contains only calculation part is called CPU burst where as the process that contains all I/O operation called I/O burst.

The processes that are placed in ready queue will be selected by short term scheduler and the selected process will be allocated to CPU by dispatcher.

When the ready queue is full by the time you want to create one process very urgently then the process that was created partially will be swapped out and placed in partially created queue by the medium term scheduler and places the process you want to execute urgently in the ready queue.

1) Process Control:

(24)

An operating system executes a variety of programs:

→ Batch System - Jobs

→ Time shared Systems - user program or tasks.

→ Batch system executes jobs, whereas a time shared system has user programs or tasks.

Process: A program in execution, process execution must progress in sequential fashion.

→ In other words, if the user can execute only one program at a time, the operating system may need to support its only one own internal programmed activities, such as memory management. In many respects, all these activities are similar so we call all of them "processes".

→ we use the terms jobs and process almost interchangeably. Even though we personally prefer the term process, much of operating system theory and terminology was developed during a time when the major activity of operating system was processing job.

→ A process includes

* program counter

* stack

* data selection.

→ As we said, process is a program in execution. A process is more than the program code, which is sometimes known as the text section. It also includes current activity as represented by the value of the program counter and the contents of the processor's registers.

A process generally also includes the process stack which

contains temporary data and a data section which contains global variables. A process may also include heap, which is memory that is dynamically allocated during process runtime.

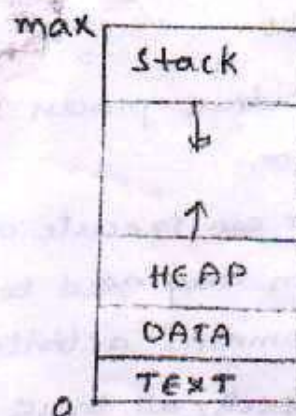


fig. 1: process in memory.

Process state:

As a process executes, it changes state. The state of a process is defined in part by the current activity of that process.

Each process may be in one of the following states.

- new : The process is being created.
- Running : Instructions are being executed.
- waiting : The process is waiting for some event to occur.
- Ready : The process is waiting to be assigned to a processor.
- terminated : The process has finished execution.

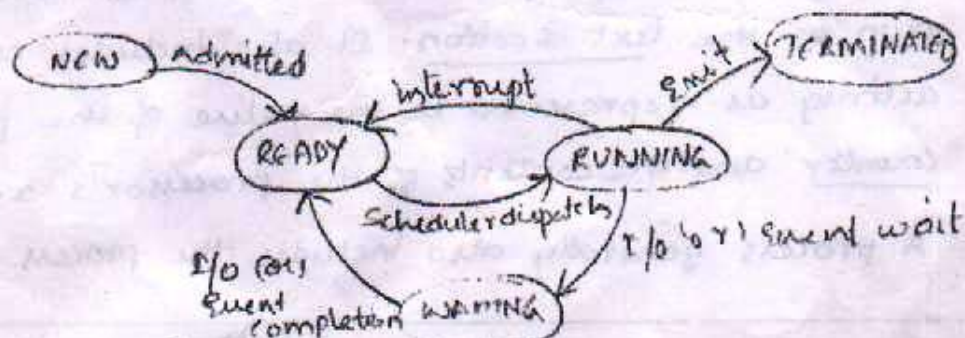


fig. 1: process state diagram

These names are arbitrary and they vary across operating systems. The states that they represent are found on all systems however, certain operating systems also more finely delineate process states. It is important to realize that only one process can be running on any processor at any instant. many processes may be ready and waiting. The state diagram of these states is given above.

Process Control Block (PCB):

Each process is represented in the operating system by a process control block, It is also called as task control block.

→ Information associated with each process:

- * process ID
- * process state
- * program counter
- * cpu registers
- * cpu scheduling information
- * memory-management information
- * Accounting information
- * I/O status information.

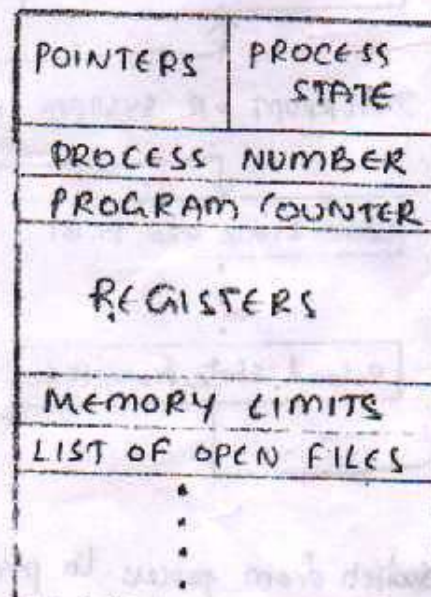


fig: process control block.

process state: The state may be new, ready, running, waiting, halted and soon.

Program Counter: The counter indicates the address of the next instruction to be executed for this process.

CPU Registers: The registers vary in number and type, depending on the computer architecture. They include accumulators, index registers, stack pointers and general purpose registers, plus any condition-code information. Along with program counter, this state information must be saved when the interrupt occurs, to allow the process to be continued correctly afterward.

cpu switch from process to process:

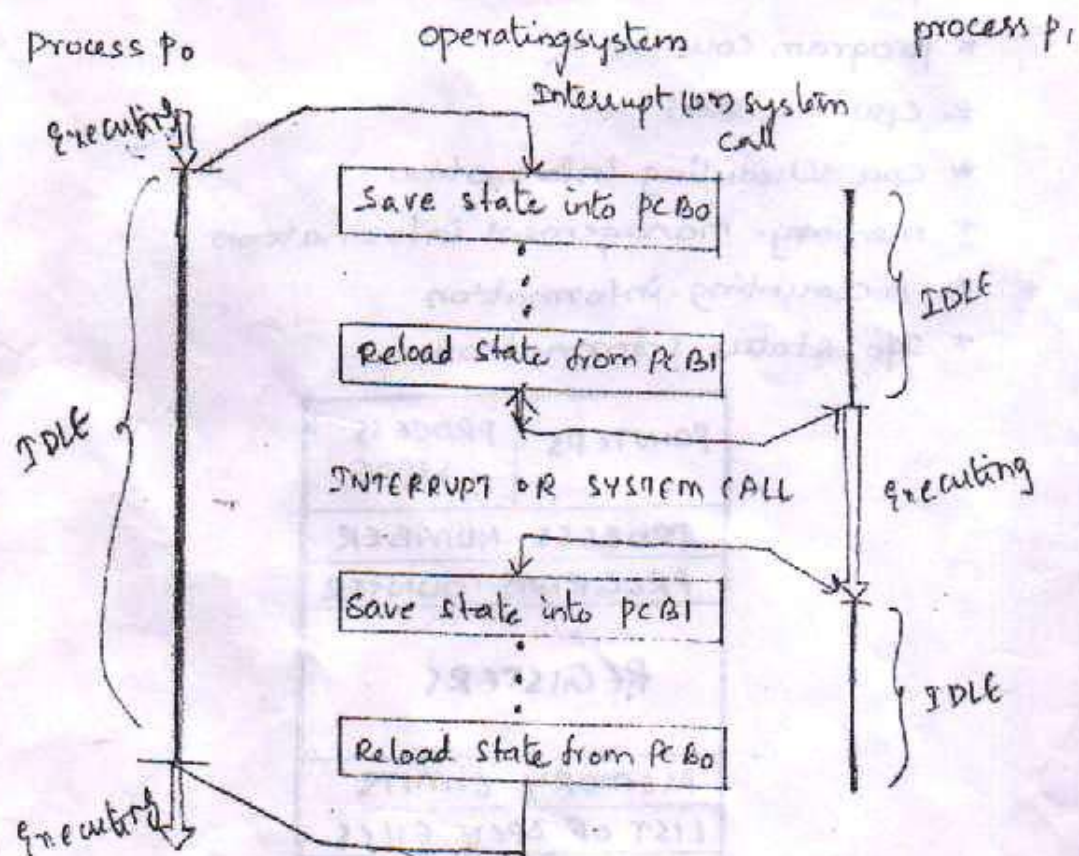


fig: CPU Switch from process to process.

Cpu-scheduling information: This information includes a process priority, pointers to scheduling queues, and any other scheduling parameters.

The different types of scheduling queues are

- * Job queue
- * ready queue
- * device queue

memory-management information: This information may include such ~~information~~ as the value of the base and limit registers, the page tables, or the segment tables, depending on the memory system used by the operating system.

Accounting information: This information includes the amount of cpu and real-time used, time limits, account numbers, job or process numbers and so on.

I/O status information: This information includes the list of I/O devices allocated to the process, a list of open files and so on.

→ PCB simply serves as the repository for any information that may vary from process to process.

Reasons for process termination:

There are certain reasons for terminating the process.

- 1) Normal completion
- 2) Unavailability of required memory.
- 3) Exceed in execution limit time.
- 4) protection error
- 5) Arithmetic error
- 6) I/O failure
- 7) misuse of data.

(29)

- 8) Exceed in waiting time limit
- 9) Invalid instruction execution
- 10) privileged instruction execution
- 11) parent process termination
- 12) parent request for child.

→ Differences between Process and Thread } Prepare on your own
→ Types or levels of Threads

* Scheduling Algorithms:-

(30)

madhu/15/11/2012

18

①

→ First-come, First-served Scheduling (FCFS):-

The simplest CPU-scheduling algorithm is the FCFS scheduling algorithm. In this algorithm the process that requests the CPU first is allocated the CPU first. The implementation of the FCFS policy is easily managed with a FIFO queue. When the process enters the ready queue, its PCB (Process Control Block) is linked to the tail of queue. When CPU is free, the PCB is allocated to the process at the head of the queue. The running process then is removed from the queue.

As an example of FCFS scheduling, consider the following set of processes, with the length of the CPU burst time given in milliseconds.

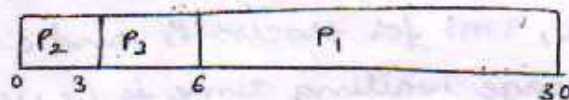
Ex:-

<u>Process</u>	<u>Burst Time</u>
P ₁	24
P ₂	3
P ₃	3

If the processes arrive in the order P₁, P₂, P₃ and are served in FCFS order, we get the result shown in the following Gantt Chart:



The waiting time is 0 milliseconds for process P₁, 24 milliseconds for process P₂, and 27 milliseconds for the process P₃. Thus, the average waiting time is $(0 + 24 + 27)/3 = 17$ milliseconds. If the processes arrive in the order P₂, P₃, P₁ however the results will be shown in the following Gantt chart:



The average waiting time is now $(6 + 0 + 3)/3 = 3$ ms.

(31)

The FCFS scheduling algorithm is nonpreemptive. Once the CPU has been allocated to a process, that process keeps the CPU until it releases the CPU, either by terminating (s) by requesting I/O.

→ Shortest-job-First Scheduling:-

A different approach to CPU scheduling is the shortest-job-first (SJF) scheduling algorithm. This algorithm associates with each process the length of the process's next CPU burst. When CPU is available, it is assigned to the process having smallest next CPU burst. If the next CPU bursts of two processes are the same, FCFS scheduling is used to break the tie. This scheduling is called shortest-next-CPU burst algorithm because scheduling depends on the length of next CPU burst of a process rather than its total length.

As an example of SJF scheduling, consider the following set of processes, with the length of CPU burst time given in milliseconds (ms):-

<u>Process</u>	<u>Burst time</u>
P ₁	6
P ₂	8
P ₃	7
P ₄	3

Using SJF scheduling, we should schedule these processes according to the following Gantt Chart:



The waiting time is 3 milliseconds for process P₁, 16 ms for process P₂, 9 ms for process P₃ and 0 ms for process P₄. Thus the average waiting time is $(3+16+9+0)/4 = 7$ ms. The SJF algorithm can be either preemptive or nonpreemptive.

(31)

The FCFS scheduling algorithm is nonpreemptive. Once the CPU has been allocated to a process, that process keeps the CPU until it releases the CPU, either by terminating (s) by requesting I/O.

→ Shortest-job-First Scheduling:-

A different approach to CPU scheduling is the shortest-job-first (SJF) scheduling algorithm. This algorithm associates with each process the length of the process's next CPU burst. When CPU is available, it is assigned to the process having smallest next CPU burst. If the next CPU bursts of two processes are the same, FCFS scheduling is used to break the tie. This scheduling is called shortest-next-CPU burst algorithm because scheduling depends on the length of next CPU burst of a process rather than its total length.

As an example of SJF scheduling, consider the following set of processes, with the length of CPU burst time given in milliseconds (ms):-

<u>Process</u>	<u>Burst time</u>
P ₁	6
P ₂	8
P ₃	7
P ₄	3

Using SJF scheduling, we should schedule these processes according to the following Gantt Chart:



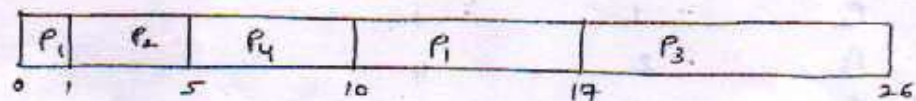
The waiting time is 3 milliseconds for process P₁, 16 ms for process P₂, 9 ms for process P₃ and 0 ms for process P₄. Thus the average waiting time is $(3+16+9+0)/4 = 7$ ms. The SJF algorithm can be either preemptive or nonpreemptive.

-ptive. The choice arises when a new process arrives at the ready queue while a previous process is still executing. The next CPU burst of the newly arrived process may be shorter than what is left of the currently executing processes. A preemptive SJF algorithm will preempt the currently executing process, whereas a non-preemptive SJF algorithm will allow the currently running process to finish its CPU burst. Preemptive SJF scheduling is sometimes called shortest-remaining-time-first scheduling.

As an example for preemptive SJF scheduling, consider the following four processes, with the length of the CPU burst given in milliseconds:

Process	Arrival time	Burst time
P ₁	0	8
P ₂	1	4
P ₃	2	9
P ₄	3	5

Then the resulting preemptive SJF schedule is depicted in the following Gantt chart:



Process P₁ is started at time 0, since it is the only process in the queue. Process P₂ arrives at time 1. The remaining time for process P₁ (7 milliseconds) is larger than the time required by process P₂ (4 milliseconds), so process P₁ is preempted, and process P₂ is scheduled. The average waiting time for the above example is $((10-1) + (1-1) + (17-2) + (5-3))/4 = 26/4 = 6.5$ milliseconds.

Nonpreemptive SJF scheduling would result in an average waiting time of 7.75 milliseconds.

→ Priority Scheduling :-

The SJF is a special case of the general priority scheduling algorithm. A priority is associated with each process, and the CPU is allocated to the process with the highest priority. Equal-priority processes are scheduled in FCFS order. As SJF algorithm is simply a priority algorithm where the priority (P) is the inverse of the next CPU burst. The larger the CPU burst, the lower the priority & vice versa.

As an example, consider the following set of processes, assumed to have arrived at time 0, in the order P_1, P_2, P_3, P_4, P_5 with the length of the CPU burst given in milliseconds:

Ex:-

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

Using priority scheduling we would schedule these processes according to the following Gantt chart:



The average waiting time is 8.2 milliseconds.

Priorities can be defined either internally & externally. Internally defined priorities use some measure

able quantity or quantities to compute the priority of a process. External priorities are set by criteria outside the operating system, such as the importance of the process, the type and amount of funds being paid for computer use, the department sponsoring the work, and other, often political, factors.

Priority scheduling can be either preemptive or nonpreemptive. A preemptive priority scheduling algorithm will preempt the CPU if the priority of newly arrived process is higher than the priority of the currently running process. A nonpreemptive priority scheduling algorithm will simply put the new process at the head of the ready queue.

A major problem with priority scheduling algorithms is indefinite blocking (or) starvation. A process that is ready to run but waiting for the CPU can be considered blocked. A priority scheduling algorithm can leave some low priority processes waiting indefinitely. In a heavily loaded computer system, a steady stream of higher priority processes can prevent a low-priority processes from getting the CPU.

→ Round-Robin Scheduling:-

The round-robin (RR) scheduling algorithm is designed especially for time-sharing systems. It is similar to FCFS scheduling, but preemption is added to switch between processes. A small unit of time, called a time quantum (or) time slice is defined. A time quantum is generally from 10 to 100 milliseconds. The ready queue

is treated as a circular queue. The CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval of upto $\pm$ time quantum.

The average waiting time under RR policy is often long. As an example for RR scheduling, consider the following processes that arrive at time 0, with the length of the CPU burst given in milliseconds:

Ex:-

Process	Burst time
P_1	24
P_2	3
P_3	3

If we use a time quantum of 4 milliseconds, then process P_1 gets the first 4 milliseconds. Since it requires another 20 milliseconds, it is preempted after the first time quantum, and the CPU is given to the next process in the queue, process P_2 . Since process P_2 does not need 4 milliseconds, it quits before its time quantum expires. The CPU is then given to the next process, process P_3 . Once each process has received $\pm$ time quantum, the CPU is returned to process P_1 for an additional time quantum. The resulting RR schedule is:-



The average waiting time is $17/3 = 5.66$ milliseconds. In RR scheduling algorithm, no process is allocated the CPU for more than $\pm$ time quantum in a row. If a process's CPU burst exceeds $\pm$ time quantum, that process is preempted & is put back in the ready queue. Thus RR is pre-emptive.